Задание 2

1. Новый файл `brain/intent_classifier.py`

```
Copy"""
IntentClassifier — семантический классификатор намерений на б
азе LLM.

Вместо regex и ключевых слов — понимание смысла через Ollama.
Включает кэширование частых паттернов для скорости.
"""

import json
import hashlib
import re
import requests
import os
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple


@dataclass
class ClassifiedIntent:
    """Результат классификации намерения."""
    intent: str                           # CREATE_WORKFLOW,
DEBUG, CLARIFY, CHAT, etc.
    confidence: float                     # 0.0 - 1.0
    entities: Dict[str, Any]              # Извлечённые сущно
сти
    missing_required: List[str]           # Чего не хватает д
ля выполнения
    clarify_question: Optional[str]       # Вопрос пользовате
```

```
лю если нужно уточнение
    original_message: str                # Исходное сообщени
е
    raw_llm_response: str = ""           # Сырой ответ LLM д
ля отладки
    from_cache: bool = False             # Был ли результат
из кэша


@dataclass
class ErrorInterpretation:
    """Результат интерпретации ошибки."""
    error_type: str                      # rate_limit, auth,
network, config, unknown
    explanation: str                     # Простое объяснени
е
    cause: str                           # Почему произошло
    solutions: List[Dict[str, Any]]      # Варианты решения
    question_to_user: Optional[str]      # Вопрос пользовате
лю
    can_auto_fix: bool                   # Есть ли автоматич
еское решение
    auto_fix_action: Optional[str]       # Какое действие дл
я авто-фикса
    original_error: str                  # Исходная ошибка


# ============================================================
==================
# Системные промпты
# ============================================================
==================

CLASSIFIER_SYSTEM_PROMPT = """Ты — классификатор намерений по
льзователя для AI-агента.
```

```
Твоя задача: понять, что хочет пользователь, и вернуть структ
урированный JSON.

ДОСТУПНЫЕ ИНТЕНТЫ (intent):

1. CREATE_WORKFLOW — создать автоматизацию / workflow / парсе
р / бота
   Триггеры: "автоматизируй", "создай workflow", "хочу чтобы
каждый день", "настрой парсинг",
   "собирай данные", "мониторь", "отслеживай", "присылай мн
е", "сделай бота"

2. DEBUG_WORKFLOW — исправить / отладить / разобраться с ошиб
кой в workflow
   Триггеры: "почему не работает", "ошибка", "исправь", "отла
дь", "сломалось", "не запускается"

3. RUN_WORKFLOW — запустить существующий workflow
   Триггеры: "запусти", "выполни", "активируй", "включи workf
low"

4. LIST_WORKFLOWS — показать список workflow / автоматизаций
   Триггеры: "покажи workflow", "список автоматизаций", "каки
е есть workflow", "мои автоматизации"

5. LIST_TEMPLATES — показать шаблоны / готовые решения
   Триггеры: "какие шаблоны", "готовые workflow", "примеры ав
томатизаций"

6. MANAGE_FILES — работа с файлами
   Триггеры: "найди файлы", "удали", "дубликаты", "очисти пап
ку", "создай файл"

7. SYSTEM_INFO — информация о системе
   Триггеры: "место на диске", "память", "CPU", "системная ин
формация"
```

```
8. WEB_SEARCH — поиск в интернете
   Триггеры: "найди в интернете", "загугли", "поищи", "что та
кое"

9. SEND_MESSAGE — отправить сообщение
   Триггеры: "отправь", "напиши", "пошли сообщение", "email",
"в телеграм"

10. OPEN_URL — открыть сайт / приложение
    Триггеры: "открой", "зайди на", "покажи сайт"

11. CHAT — разговор, вопрос, благодарность (НЕ требует действ
ий)
    Триггеры: "привет", "спасибо", "как дела", "что умеешь",
"кто ты"

12. PROVIDE_INFO — пользователь даёт информацию (ответ на воп
рос агента)
    Триггеры: сообщение содержит @каналы, токены, API ключи,
URLs, числа
    Используй когда сообщение выглядит как ОТВЕТ, а не как но
вый запрос

13. CONFIRM — подтверждение действия
    Триггеры: "да", "ок", "давай", "подтверждаю", "согласен",
"вариант 1"

14. CANCEL — отмена действия
    Триггеры: "нет", "отмена", "стоп", "не надо", "другое"

СУЩНОСТИ (entities) для извлечения:

- source: откуда данные (telegram, rss, website, email, twitt
er, instagram)
- channels: список каналов [@channel1, @channel2] — извлекай
```

```
ВСЕ что начинается с @
- schedule: расписание (daily, hourly, weekly, "в 11 утра",
"каждый час")
- schedule_time: конкретное время если указано ("11:00", "утр
ом")
- destination: куда результат (telegram, email, file, notion)
- path: путь к файлу/папке
- url: URL сайта
- workflow_name: название workflow если упомянуто
- workflow_id: ID workflow если упомянуто
- query: поисковый запрос
- credentials: токены/ключи если переданы (НЕ парси, просто о
тметь has_credentials: true)
- format: формат вывода если указан ("название - описание",
"кратко", "подробно")
- topic: тема/фильтр ("AI сервисы", "новости", "обновления")

ПРАВИЛА:

1. Извлекай ВСЕ @упоминания в channels, даже если их много
2. Если пользователь отвечает на вопрос агента — это PROVIDE_
INFO, не новый запрос
3. Если не хватает критичной информации — заполни missing_req
uired и clarify_question
4. confidence: 0.9+ уверен, 0.7-0.9 вероятно, <0.7 угадываю
5. Для CREATE_WORKFLOW всегда нужны: source, channels (если s
ource=telegram)
6. Отвечай ТОЛЬКО JSON, без markdown

ФОРМАТ ОТВЕТА:
{
  "intent": "CREATE_WORKFLOW",
  "confidence": 0.95,
  "entities": {
    "source": "telegram",
    "channels": ["@channel1", "@channel2"],
```

```
    "schedule": "daily",
    "schedule_time": "11:00",
    "destination": "telegram",
    "topic": "AI новости",
    "format": "название - описание"
  },
  "missing_required": [],
  "clarify_question": null
}"""


ERROR_INTERPRETER_PROMPT = """Ты — эксперт по анализу ошибок.
Объясни ошибку ПРОСТЫМ языком.

ЗАДАЧА ПОЛЬЗОВАТЕЛЯ:
{task}

КОНТЕКСТ:
- Сервис: {service}
- Количество каналов/элементов: {item_count}
- Параметры: {params}

ОШИБКА:
{error}

Проанализируй и верни JSON:
{{
  "error_type": "rate_limit | auth | network | config | data
| unknown",
  "explanation": "Что случилось — 1 простое предложение",
  "cause": "Почему это произошло в контексте этой задачи",
  "solutions": [
    {{
      "title": "Название решения",
      "description": "Что сделать — понятно для нетехническог
о человека",
```

```
      "can_auto_fix": true,
      "auto_fix_action": "reduce_items | add_delay | retry |
use_local | null"
    }}
  ],
  "question_to_user": "Вопрос если нужен выбор, иначе null"
}}

ТИПЫ ОШИБОК:
- rate_limit: 403, 429, "too many requests", "forbidden" — ли
миты API
- auth: 401, "unauthorized", "credentials" — проблемы с досту
пом
- network: timeout, connection refused — сеть недоступна
- config: 400, "invalid", "missing parameter" — неправильные
настройки
- data: 404, "not found" — данные не найдены
- unknown: всё остальное

АВТО-ФИКСЫ:
- reduce_items: уменьшить количество каналов/элементов до 5
- add_delay: добавить паузы между запросами
- retry: просто повторить через минуту
- use_local: предложить локальный сервис вместо публичного

ВАЖНО:
- Пиши на русском
- Первым ставь решение которое можно сделать автоматически
- Не пугай пользователя техническими деталями"""


# ==========================================================
==================
# IntentClassifier
# ==========================================================
==================
```

```
class IntentClassifier:
    """Классификатор намерений с LLM и кэшированием."""

    # Кэш: normalized_message -> (ClassifiedIntent, timestam
p)
    _cache: Dict[str, Tuple[ClassifiedIntent, datetime]] = {}
    _cache_ttl: timedelta = timedelta(hours=24)
    _cache_file: Optional[Path] = None

    def __init__(
        self,
        ollama_url: str = None,
        model: str = None,
        timeout: int = 45,
        cache_dir: Optional[Path] = None,):
        self.ollama_url = ollama_url or os.environ.get("OLLAM
A_URL", "http://localhost:11434")
        self.model = model or os.environ.get("OLLAMA_MODEL",
"qwen2.5-coder:14b")
        self.timeout = timeout

        if cache_dir:
            self._cache_file = cache_dir / "intent_cache.jso
n"
            self._load_cache()

    def classify(
        self,
        user_message: str,
        context: Optional[Dict[str, Any]] = None) -> Classifi
edIntent:
        """
        Классифицирует намерение пользователя.

        Args:
```

```
            user_message: Сообщение пользователя
            context: Контекст (last_error, last_workflow, pen
ding_task)

        Returns:
            ClassifiedIntent
        """
        # Быстрые проверки без LLM
        quick_result = self._quick_classify(user_message)
        if quick_result:
            return quick_result

        # Проверяем кэш
        cache_key = self._cache_key(user_message)
        cached = self._get_from_cache(cache_key)
        if cached:
            cached.original_message = user_message
            cached.from_cache = True
            return cached

        # Вызываем LLM
        result = self._classify_with_llm(user_message, contex
t)

        # Кэшируем если уверены
        if result.confidence >= 0.85:
            self._add_to_cache(cache_key, result)

        return result

    def _quick_classify(self, message: str) -> Optional[Class
ifiedIntent]:
        """Быстрая классификация без LLM для очевидных случае
в."""
        msg = message.strip().lower()
```

```
        # Подтверждения
        if msg in ("да", "yes", "ок", "ok", "давай", "го",
"1", "подтверждаю"):
            return ClassifiedIntent(
                intent="CONFIRM",
                confidence=0.99,
                entities={"choice": msg},
                missing_required=[],
                clarify_question=None,
                original_message=message,
                from_cache=False,
            )

        # Отмены
        if msg in ("нет", "no", "отмена", "cancel", "стоп",
"не надо"):
            return ClassifiedIntent(
                intent="CANCEL",
                confidence=0.99,
                entities={},
                missing_required=[],
                clarify_question=None,
                original_message=message,
                from_cache=False,
            )

        # Выбор варианта по номеру
        if re.match(r'^[1-9]$|^вариант\s*[1-9]$', msg):
            num = re.search(r'[1-9]', msg).group()
            return ClassifiedIntent(
                intent="CONFIRM",
                confidence=0.99,
                entities={"choice": int(num)},
                missing_required=[],
                clarify_question=None,
                original_message=message,
```

```
                from_cache=False,
            )

        # Сообщение с кучей @каналов — скорее всего ответ с д
анными
        channels = re.findall(r'@[\w]+', message)
        if len(channels) >= 3:
            return ClassifiedIntent(
                intent="PROVIDE_INFO",
                confidence=0.95,
                entities={"channels": channels},
                missing_required=[],
                clarify_question=None,
                original_message=message,
                from_cache=False,
            )

        # Сообщение с токеном бота
        if re.search(r'\d{8,10}:[A-Za-z0-9_-]{35}', message):
            return ClassifiedIntent(
                intent="PROVIDE_INFO",
                confidence=0.99,
                entities={"has_credentials": True, "credentia
l_type": "telegram_bot_token"},
                missing_required=[],
                clarify_question=None,
                original_message=message,
                from_cache=False,
            )

        return None

    def _classify_with_llm(
        self,
        user_message: str,
        context: Optional[Dict[str, Any]]) -> ClassifiedInten
```

```
t:
        """Классификация через Ollama."""
        # Формируем контекст
        context_parts = []
        if context:
            if context.get("last_error"):
                context_parts.append(f"Предыдущая ошибка: {co
ntext['last_error'][:200]}")
            if context.get("last_workflow"):
                context_parts.append(f"Последний workflow: {c
ontext['last_workflow']}")
            if context.get("pending_task"):
                context_parts.append(f"Ожидает ответа: {conte
xt['pending_task']}")
            if context.get("awaiting"):
                context_parts.append(f"Агент ждёт: {context
['awaiting']}")

        context_str = "\n".join(context_parts) if context_par
ts else "Нет контекста"

        user_prompt = f"""Контекст диалога:
{context_str}

Сообщение пользователя:
{user_message}"""

        try:
            resp = requests.post(
                f"{self.ollama_url}/api/chat",
                json={
                    "model": self.model,
                    "messages": [
                        {"role": "system", "content": CLASSIF
IER_SYSTEM_PROMPT},
                        {"role": "user", "content": user_prom
```

```
pt}
                    ],
                    "stream": False,
                    "options": {"temperature": 0.1, "num_pred
ict": 1500}
                },
                timeout=self.timeout
            )
            resp.raise_for_status()
            answer = resp.json().get("message", {}).get("cont
ent", "")
            parsed = self._parse_json(answer)

            # Дополнительно извлекаем каналы если LLM пропуст
ил
            if not parsed.get("entities", {}).get("channel
s"):
                channels = re.findall(r'@[\w]+', user_messag
e)
                if channels:
                    parsed.setdefault("entities", {})["channe
ls"] = channels

            return ClassifiedIntent(
                intent=parsed.get("intent", "CHAT"),
                confidence=parsed.get("confidence", 0.5),
                entities=parsed.get("entities", {}),
                missing_required=parsed.get("missing_require
d", []),
                clarify_question=parsed.get("clarify_questio
n"),
                original_message=user_message,
                raw_llm_response=answer,
                from_cache=False,
            )
```

```
        except requests.exceptions.Timeout:
            return self._fallback_classification(user_messag
e, "LLM timeout")
        except requests.exceptions.ConnectionError:
            return self._fallback_classification(user_messag
e, "LLM unavailable")
        except Exception as e:
            return self._fallback_classification(user_messag
e, str(e))

    def _fallback_classification(self, message: str, error: s
tr) -> ClassifiedIntent:
        """Fallback когда LLM недоступен — простая эвристик
a."""
        msg = message.lower()

        # Простые паттерны
        if any(w in msg for w in ["создай", "автоматиз", "wor
kflow", "настрой", "хочу чтобы"]):
            intent = "CREATE_WORKFLOW"
        elif any(w in msg for w in ["ошибка", "исправь", "поч
ему не", "debug", "отладь"]):
            intent = "DEBUG_WORKFLOW"
        elif any(w in msg for w in ["покажи", "список", "каки
е есть"]):
            intent = "LIST_WORKFLOWS"
        elif any(w in msg for w in ["найди", "поиск", "загугл
и"]):
            intent = "WEB_SEARCH"
        elif any(w in msg for w in ["открой", "зайди"]):
            intent = "OPEN_URL"
        else:
            intent = "CHAT"

        channels = re.findall(r'@[\w]+', message)
```

```
        return ClassifiedIntent(
            intent=intent,
            confidence=0.5,
            entities={"channels": channels} if channels else
{},
            missing_required=[],
            clarify_question=None,
            original_message=message,
            raw_llm_response=f"Fallback due to: {error}",
            from_cache=False,
        )

    def _cache_key(self, message: str) -> str:
        """Генерирует ключ кэша для сообщения."""
        # Нормализуем: lowercase, убираем лишние пробелы, сор
тируем @каналы
        normalized = message.lower().strip()
        normalized = re.sub(r'\s+', ' ', normalized)

        # Заменяем конкретные каналы на плейсхолдер (чтобы кэ
ш работал для разных каналов)
        normalized = re.sub(r'@[\w]+', '@CHANNEL', normalize
d)

        return hashlib.md5(normalized.encode()).hexdigest()[:
16]

    def _get_from_cache(self, key: str) -> Optional[Classifie
dIntent]:
        """Получает из кэша если не протух."""
        if key not in self._cache:
            return None

        cached, timestamp = self._cache[key]
        if datetime.now() - timestamp > self._cache_ttl:
            del self._cache[key]
```

```
            return None

        return ClassifiedIntent(
            intent=cached.intent,
            confidence=cached.confidence,
            entities=dict(cached.entities),
            missing_required=list(cached.missing_required),
            clarify_question=cached.clarify_question,
            original_message=cached.original_message,
            raw_llm_response=cached.raw_llm_response,
            from_cache=True,
        )

    def _add_to_cache(self, key: str, result: ClassifiedInten
t) -> None:
        """Добавляет в кэш."""
        self._cache[key] = (result, datetime.now())
        self._save_cache()

    def _load_cache(self) -> None:
        """Загружает кэш из файла."""
        if not self._cache_file or not self._cache_file.exist
s():
            return
        try:
            data = json.loads(self._cache_file.read_text(enco
ding="utf-8"))
            for key, item in data.items():
                timestamp = datetime.fromisoformat(item["time
stamp"])
                if datetime.now() - timestamp < self._cache_t
tl:
                    self._cache[key] = (
                        ClassifiedIntent(
                            intent=item["intent"],
                            confidence=item["confidence"],
```

```
                            entities=item["entities"],
                            missing_required=item["missing_re
quired"],
                            clarify_question=item.get("clarif
y_question"),
                            original_message=item.get("origin
al_message", ""),
                            raw_llm_response="",
                            from_cache=True,
                        ),
                        timestamp
                    )
        except Exception:
            pass

    def _save_cache(self) -> None:
        """Сохраняет кэш в файл."""
        if not self._cache_file:
            return
        try:
            data = {}
            for key, (result, timestamp) in self._cache.items
():
                data[key] = {
                    "intent": result.intent,
                    "confidence": result.confidence,
                    "entities": result.entities,
                    "missing_required": result.missing_requir
ed,
                    "clarify_question": result.clarify_questi
on,
                    "original_message": result.original_messa
ge,
                    "timestamp": timestamp.isoformat(),
                }
            self._cache_file.parent.mkdir(parents=True, exist
```

```
_ok=True)
            self._cache_file.write_text(json.dumps(data, ensu
re_ascii=False, indent=2), encoding="utf-8")
        except Exception:
            pass

    def _parse_json(self, text: str) -> Dict:
        """Парсит JSON из ответа LLM."""
        text = text.strip()
        if "```json" in text:
            text = text.split("```json")[1].split("```")[0]
        elif "```" in text:
            parts = text.split("```")
            if len(parts) >= 2:
                text = parts[1]

        start = text.find("{")
        end = text.rfind("}") + 1
        if start != -1 and end > start:
            try:
                return json.loads(text[start:end])
            except json.JSONDecodeError:
                pass
        return {}


# ============================================================
==================
# SmartErrorInterpreter
# ============================================================
==================

class SmartErrorInterpreter:
    """Интерпретатор ошибок на базе LLM."""

    def __init__(
```

```
        self,
        ollama_url: str = None,
        model: str = None,
        timeout: int = 30,):
        self.ollama_url = ollama_url or os.environ.get("OLLAM
A_URL", "http://localhost:11434")
        self.model = model or os.environ.get("OLLAMA_MODEL",
"qwen2.5-coder:14b")
        self.timeout = timeout

    def interpret(
        self,
        error: str,
        context: Dict[str, Any]) -> ErrorInterpretation:
        """
        Интерпретирует ошибку через LLM.

        Args:
            error: Текст ошибки
            context: task, service, channels, params

        Returns:
            ErrorInterpretation
        """
        # Подсчёт элементов
        item_count = 0
        channels = context.get("params", {}).get("channels",
[])
        if isinstance(channels, list):
            item_count = len(channels)
        elif isinstance(channels, str):
            item_count = len(re.findall(r'@[\w]+', channels))

        prompt = ERROR_INTERPRETER_PROMPT.format(
            task=context.get("task", "Не указана"),
            service=context.get("service", "unknown"),
```

```
            item_count=item_count,
            params=json.dumps(context.get("params", {}), ensu
re_ascii=False)[:500],
            error=error[:1000]
        )

        try:
            resp = requests.post(
                f"{self.ollama_url}/api/chat",
                json={
                    "model": self.model,
                    "messages": [{"role": "user", "content":
prompt}],
                    "stream": False,
                    "options": {"temperature": 0.2, "num_pred
ict": 2000}
                },
                timeout=self.timeout
            )
            resp.raise_for_status()
            answer = resp.json().get("message", {}).get("cont
ent", "")
            parsed = self._parse_json(answer)

            solutions = parsed.get("solutions", [])
            can_auto = any(s.get("can_auto_fix") for s in sol
utions)
            auto_action = None
            for s in solutions:
                if s.get("can_auto_fix"):
                    auto_action = s.get("auto_fix_action")
                    break

            return ErrorInterpretation(
                error_type=parsed.get("error_type", "unknow
n"),
```

```
                explanation=parsed.get("explanation", f"Произ
ошла ошибка: {error[:100]}"),
                cause=parsed.get("cause", "Не удалось определ
ить причину"),
                solutions=solutions,
                question_to_user=parsed.get("question_to_use
r"),
                can_auto_fix=can_auto,
                auto_fix_action=auto_action,
                original_error=error,
            )

        except Exception as e:
            return self._fallback_interpretation(error, conte
xt, str(e))

    def _fallback_interpretation(
        self,
        error: str,
        context: Dict[str, Any],
        llm_error: str) -> ErrorInterpretation:
        """Fallback интерпретация без LLM."""
        error_lower = error.lower()

        # Определяем тип по паттернам
        if any(x in error_lower for x in ["403", "429", "rat
e", "too many", "forbidden"]):
            error_type = "rate_limit"
            explanation = "Сервис заблокировал запрос — слишк
ом много обращений"
            cause = "Публичные API ограничивают количество за
просов"
            solutions = [
                {
                    "title": "Уменьшить количество",
                    "description": "Начну с 5 каналов, потом
```

```
добавим остальные",
                    "can_auto_fix": True,
                    "auto_fix_action": "reduce_items"
                },
                {
                    "title": "Добавить паузы",
                    "description": "Делать запросы с задержко
й 2-3 секунды",
                    "can_auto_fix": True,
                    "auto_fix_action": "add_delay"
                }
            ]
        elif any(x in error_lower for x in ["401", "unauthori
zed", "credentials", "auth"]):
            error_type = "auth"
            explanation = "Нет доступа — нужны учётные данны
е"
            cause = "Сервис требует авторизацию"
            solutions = [
                {
                    "title": "Добавить API ключ",
                    "description": "Нужен токен или ключ для
доступа к сервису",
                    "can_auto_fix": False,
                    "auto_fix_action": None
                }
            ]
        elif any(x in error_lower for x in ["timeout", "timed
out", "connection"]):
            error_type = "network"
            explanation = "Сервис не отвечает"
            cause = "Проблемы с сетью или сервис перегружен"
            solutions = [
                {
                    "title": "Повторить позже",
                    "description": "Подожду минуту и попробую
```

```
снова",
                    "can_auto_fix": True,
                    "auto_fix_action": "retry"
                }
            ]
        else:
            error_type = "unknown"
            explanation = f"Произошла ошибка: {error[:100]}"
            cause = "Не удалось определить причину автоматиче
ски"
            solutions = [
                {
                    "title": "Попробовать иначе",
                    "description": "Опиши задачу по-другому",
                    "can_auto_fix": False,
                    "auto_fix_action": None
                }
            ]

        return ErrorInterpretation(
            error_type=error_type,
            explanation=explanation,
            cause=cause,
            solutions=solutions,
            question_to_user="Какой вариант выбираешь?",
            can_auto_fix=any(s["can_auto_fix"] for s in solut
ions),
            auto_fix_action=next((s["auto_fix_action"] for s
in solutions if s["can_auto_fix"]), None),
            original_error=error,
        )

    def format_for_user(self, interpretation: ErrorInterpreta
tion) -> str:
        """Форматирует в читаемое сообщение."""
        lines = [interpretation.explanation, ""]
```

```
        if interpretation.cause:
            lines.append(f"Причина: {interpretation.cause}")
            lines.append("")

        if interpretation.solutions:
            lines.append("Варианты решения:")
            for i, sol in enumerate(interpretation.solutions,
1):
                auto_mark = " ✓" if sol.get("can_auto_fix") e
lse ""
                lines.append(f"{i}. **{sol['title']}**{auto_m
ark}")
                lines.append(f"   {sol['description']}")
            lines.append("")

        if interpretation.question_to_user:
            lines.append(interpretation.question_to_user)
        elif interpretation.can_auto_fix:
            lines.append("Попробовать вариант 1 автоматическ
и? (да/нет)")

        return "\n".join(lines)

    def _parse_json(self, text: str) -> Dict:
        """Парсит JSON."""
        text = text.strip()
        if "```json" in text:
            text = text.split("```json")[1].split("```")[0]
        elif "```" in text:
            parts = text.split("```")
            if len(parts) >= 2:
                text = parts[1]

        start = text.find("{")
        end = text.rfind("}") + 1
```

```
        if start != -1 and end > start:
            try:
                return json.loads(text[start:end])
            except:
                pass
        return {}
Copy
```

2. Обновлённый `brain/brain_layer.py`

Теперь использует `IntentClassifier` вместо regex:

```
Copy"""BrainLayer — orchestrator с семантическим классификато
ром намерений."""

from __future__ import annotations

import re
from datetime import date
from pathlib import Path
from typing import Any, Dict, List, Optional

from brain.executor import Executor, StepResult
from brain.planner import PlanStep, Planner
from brain.router import Router
from brain.verifier import VerificationResult, Verifier
from brain.intent_classifier import IntentClassifier, SmartEr
rorInterpreter, ClassifiedIntent, ErrorInterpretation


class BrainLayer:
    """Orchestrate User → Classify → Route → Execute → Verif
y."""

    def __init__(
```

```
        self,
        controller: Any,
        *,
        require_confirmation: bool = False,
        memory_dir: Optional[Path] = None,) -> None:
        self.controller = controller
        self.router = Router()
        self.planner = Planner()
        self.executor = Executor(controller)
        self.verifier = Verifier()

        # Семантические классификаторы
        cache_dir = memory_dir if memory_dir else Path(__file
__).parent.parent / "memory"
        self.intent_classifier = IntentClassifier(cache_dir=c
ache_dir)
        self.error_interpreter = SmartErrorInterpreter()

        # State
        self._pending: Optional[Dict] = None
        self._require_confirmation = require_confirmation

        # Learned rules
        self._rules_file = Path(__file__).parent / "learned_r
ules.md"
        self._learned_rules: List[str] = []
        self._load_learned_rules()

    # ---------------------------------------------------------
-----------
    # Public API
    # ---------------------------------------------------------
-----------

    def handle(self, user_message: str) -> Dict[str, Any]:
        """Main entry point."""
```

```
        # Step 0: Handle pending state (waiting for user resp
onse)
        if self._pending is not None:
            return self._handle_pending(user_message)

        # Step 1: Classify intent through LLM
        context = self._get_context()
        classified = self.intent_classifier.classify(user_mes
sage, context)

        # Step 2: Route by intent
        return self._route_by_intent(classified)

    # ---------------------------------------------------------
-----------
    # Intent Routing
    # ---------------------------------------------------------
-----------

    def _route_by_intent(self, classified: ClassifiedIntent)
-> Dict[str, Any]:
        """Route to appropriate handler based on classified i
ntent."""
        intent = classified.intent

        # Handle clarification needed
        if classified.missing_required and classified.clarify
_question:
            return self._ask_clarification(classified)

        # Route to handlers
        if intent == "CONFIRM":
            return self._handle_confirm(classified)

        if intent == "CANCEL":
```

```
            return self._handle_cancel(classified)

        if intent == "PROVIDE_INFO":
            return self._handle_provided_info(classified)

        if intent == "CHAT":
            return self._handle_chat(classified)

        if intent in ("CREATE_WORKFLOW", "DEBUG_WORKFLOW", "R
UN_WORKFLOW",
                      "LIST_WORKFLOWS", "LIST_TEMPLATES"):
            return self._handle_n8n_intent(classified)

        if intent in ("MANAGE_FILES", "SYSTEM_INFO", "WEB_SEA
RCH",
                      "SEND_MESSAGE", "OPEN_URL"):
            return self._handle_tool_intent(classified)

        # Fallback: let controller try
        return self._delegate_to_controller(classified.origin
al_message)

    def _ask_clarification(self, classified: ClassifiedInten
t) -> Dict[str, Any]:
        """Ask user for missing information."""
        self._pending = {
            "stage": "awaiting_clarification",
            "classified": classified,
            "missing": classified.missing_required,
        }

        return {
            "path": "CLARIFY",
            "handled": True,
            "response": classified.clarify_question,
            "tool_name": "intent_clarify",
```

```
            "tool_result": {
                "intent": classified.intent,
                "confidence": classified.confidence,
                "missing": classified.missing_required,
                "entities_so_far": classified.entities,
            },
            "awaiting_clarification": True,
        }

    def _handle_confirm(self, classified: ClassifiedIntent) -
> Dict[str, Any]:
        """Handle confirmation (да, ок, 1, etc.)"""
        # If nothing pending, treat as chat
        if not self._pending:
            return self._handle_chat(classified)

        stage = self._pending.get("stage", "")

        if stage == "awaiting_solution":
            return self._execute_pending_solution(classified.
entities.get("choice", 1))

        if stage == "awaiting_confirmation":
            return self._execute_pending_plan()

        return self._handle_chat(classified)

    def _handle_cancel(self, classified: ClassifiedIntent) ->
Dict[str, Any]:
        """Handle cancellation."""
        self._pending = None
        return {
            "path": "FAST",
            "handled": True,
            "response": "Хорошо, отменено. Чем могу помочь?",
            "tool_name": "cancelled",
```

```
            "tool_result": None,
        }

    def _handle_provided_info(self, classified: ClassifiedInt
ent) -> Dict[str, Any]:
        """Handle when user provides requested informatio
n."""
        if not self._pending:
            # No context — treat as new request
            return self._delegate_to_controller(classified.or
iginal_message)

        stage = self._pending.get("stage", "")

        if stage == "awaiting_clarification":
            # Merge new entities with existing
            original_classified: ClassifiedIntent = self._pen
ding["classified"]
            merged_entities = {**original_classified.entitie
s, **classified.entities}

            # Extract channels from the new message
            new_channels = classified.entities.get("channel
s", [])
            if new_channels:
                existing_channels = merged_entities.get("chan
nels", [])
                if isinstance(existing_channels, list):
                    merged_entities["channels"] = existing_ch
annels + new_channels
                else:
                    merged_entities["channels"] = new_channel
s

            # Create updated classification
            updated = ClassifiedIntent(
```

```
                intent=original_classified.intent,
                confidence=original_classified.confidence,
                entities=merged_entities,
                missing_required=[],  # Assume info was provi
ded
                clarify_question=None,
                original_message=original_classified.original
_message + " " + classified.original_message,
            )

            self._pending = None
            return self._route_by_intent(updated)

        if stage == "awaiting_credentials":
            # User provided credentials
            return self._retry_with_credentials(classified)

        return self._delegate_to_controller(classified.origin
al_message)

    def _handle_chat(self, classified: ClassifiedIntent) -> D
ict[str, Any]:
        """Handle conversational messages."""
        # Delegate to controller/Ollama for chat
        return self._delegate_to_controller(classified.origin
al_message)

    def _handle_n8n_intent(self, classified: ClassifiedInten
t) -> Dict[str, Any]:
        """Handle n8n-related intents."""
        intent = classified.intent
        entities = classified.entities

        # Build request for controller with extracted entitie
s
        if intent == "CREATE_WORKFLOW":
```

```
            # Enrich the message with structured data
            return self._create_workflow(classified)

        if intent == "LIST_WORKFLOWS":
            return self._delegate_to_controller("покажи все n
8n workflows")

        if intent == "LIST_TEMPLATES":
            return self._delegate_to_controller("покажи досту
пные шаблоны n8n workflow")

        if intent == "DEBUG_WORKFLOW":
            workflow = entities.get("workflow_name") or entit
ies.get("workflow_id", "")
            return self._delegate_to_controller(f"отладь n8n
workflow {workflow}")

        if intent == "RUN_WORKFLOW":
            workflow = entities.get("workflow_name") or entit
ies.get("workflow_id", "")
            return self._delegate_to_controller(f"запусти n8n
workflow {workflow}")

        return self._delegate_to_controller(classified.origin
al_message)

    def _handle_tool_intent(self, classified: ClassifiedInten
t) -> Dict[str, Any]:
        """Handle tool-based intents (files, system, web, et
c.)"""
        # Delegate to controller with original message
        return self._delegate_to_controller(classified.origin
al_message)

    def _create_workflow(self, classified: ClassifiedIntent)
-> Dict[str, Any]:
```

```
        """Create workflow from classified intent."""
        entities = classified.entities

        # Build structured request
        source = entities.get("source", "")
        channels = entities.get("channels", [])
        schedule = entities.get("schedule", "")
        schedule_time = entities.get("schedule_time", "")
        destination = entities.get("destination", "telegram")
        topic = entities.get("topic", "")
        output_format = entities.get("format", "")

        # Format channels
        if isinstance(channels, list):
            channels_str = ", ".join(channels)
        else:
            channels_str = str(channels)

        # Build natural language request for controller
        request_parts = ["создай n8n workflow:"]

        if source == "telegram" and channels_str:
            request_parts.append(f"собирать посты из телеграм
каналов {channels_str}")
        elif source:
            request_parts.append(f"источник: {source}")

        if schedule:
            time_part = f" в {schedule_time}" if schedule_tim
e else ""
            request_parts.append(f"расписание: {schedule}{tim
e_part}")

        if topic:
            request_parts.append(f"тема: {topic}")
```

```
        if output_format:
            request_parts.append(f"формат: {output_format}")

        if destination:
            request_parts.append(f"отправлять в: {destinatio
n}")

        structured_request = " | ".join(request_parts)

        # Delegate to controller with enriched request
        result = self.controller.handle_request(structured_re
quest)

        # Check for errors
        if self._is_error_result(result):
            return self._handle_error(result, classified)

        return self._wrap_result(result, classified)

    # ---------------------------------------------------------
-----------
    # Error Handling
    # ---------------------------------------------------------
-----------

    def _is_error_result(self, result: Dict[str, Any]) -> boo
l:
        """Check if result is an error that needs interpretat
ion."""
        if result.get("success") is False:
            return True

        response = str(result.get("response", "")).lower()
        error_markers = ["error", "failed", "forbidden", "sto
pped", "cannot fix", "не удалось"]
        success_markers = ["успешно", "создан", "готов", "com
```

```
pleted"]

        has_error = any(m in response for m in error_markers)
        has_success = any(m in response for m in success_mark
ers)

        return has_error and not has_success

    def _handle_error(
        self,
        result: Dict[str, Any],
        classified: ClassifiedIntent) -> Dict[str, Any]:
        """Interpret error and suggest solutions."""
        # Extract error
        error_msg = self._extract_error(result)

        # Build context
        context = {
            "task": classified.original_message,
            "service": classified.entities.get("source", "unk
nown"),
            "params": classified.entities,
        }

        # Interpret through LLM
        interpretation = self.error_interpreter.interpret(err
or_msg, context)
        formatted = self.error_interpreter.format_for_user(in
terpretation)

        # Save pending state for solution selection
        if interpretation.can_auto_fix:
            self._pending = {
                "stage": "awaiting_solution",
                "interpretation": interpretation,
                "classified": classified,
```

```
                "original_result": result,
            }

        return {
            "path": "FAST",
            "handled": True,
            "response": formatted,
            "tool_name": "error_analysis",
            "tool_result": {
                "error_type": interpretation.error_type,
                "can_auto_fix": interpretation.can_auto_fix,
                "auto_fix_action": interpretation.auto_fix_ac
tion,
                "solutions_count": len(interpretation.solutio
ns),
            },
            "awaiting_solution": True,
        }

    def _extract_error(self, result: Dict[str, Any]) -> str:
        """Extract error message from result."""
        if result.get("error"):
            return str(result["error"])

        response = str(result.get("response", ""))

        # Look for error patterns
        for pattern in [r"msg=([^\n]+)", r"error[=:]([^\n]
+)", r"reason=([^\n]+)"]:
            match = re.search(pattern, response, re.IGNORECAS
E)
            if match:
                return match.group(1)

        return response[:500]
```

```
    def _execute_pending_solution(self, choice: int = 1) -> D
ict[str, Any]:
        """Execute the chosen solution."""
        if not self._pending or self._pending.get("stage") !=
"awaiting_solution":
            return self._handle_chat(ClassifiedIntent(
                intent="CHAT", confidence=1.0, entities={}, m
issing_required=[],
                clarify_question=None, original_message="да"
            ))

        interpretation: ErrorInterpretation = self._pending
["interpretation"]
        classified: ClassifiedIntent = self._pending["classif
ied"]

        # Get solution
        solutions = interpretation.solutions
        idx = min(choice - 1, len(solutions) - 1) if solution
s else 0
        solution = solutions[idx] if solutions else None

        self._pending = None

        if not solution:
            return {
                "path": "FAST",
                "handled": True,
                "response": "Не нашёл подходящего решения. По
пробуй описать задачу иначе.",
                "tool_name": "no_solution",
            }

        if not solution.get("can_auto_fix"):
            return {
                "path": "FAST",
```

```
                "handled": True,
                "response": f"Для этого варианта нужно твоё у
частие:\n\n{solution['description']}",
                "tool_name": "solution_manual",
            }

        # Execute auto-fix
        action = solution.get("auto_fix_action")

        if action == "reduce_items":
            return self._retry_with_reduced_items(classified)

        if action == "add_delay":
            return self._retry_with_delay(classified)

        if action == "retry":
            return self._retry_original(classified)

        return {
            "path": "FAST",
            "handled": True,
            "response": f"Применяю решение: {solution['titl
e']}\n\n{solution['description']}",
            "tool_name": "solution_applied",
        }

    def _retry_with_reduced_items(self, classified: Classifie
dIntent) -> Dict[str, Any]:
        """Retry with fewer channels."""
        entities = classified.entities.copy()
        channels = entities.get("channels", [])

        if isinstance(channels, list) and len(channels) > 5:
            entities["channels"] = channels[:5]
            remaining = len(channels) - 5
```

```
            updated = ClassifiedIntent(
                intent=classified.intent,
                confidence=classified.confidence,
                entities=entities,
                missing_required=[],
                clarify_question=None,
                original_message=classified.original_message,
            )

            result = self._route_by_intent(updated)

            if not self._is_error_result(result):
                result["response"] = (
                    f"Создал workflow для первых 5 каналов. "
                    f"Осталось добавить ещё {remaining}.\n\n"
                    + result.get("response", "")
                )

            return result

        return self._delegate_to_controller(classified.origin
al_message)

    def _retry_with_delay(self, classified: ClassifiedIntent)
-> Dict[str, Any]:
        """Retry with delay hint."""
        # Add delay hint to entities
        entities = classified.entities.copy()
        entities["add_delay"] = True

        updated = ClassifiedIntent(
            intent=classified.intent,
            confidence=classified.confidence,
            entities=entities,
            missing_required=[],
            clarify_question=None,
```

```
            original_message=classified.original_message,
        )

        return self._route_by_intent(updated)

    def _retry_original(self, classified: ClassifiedIntent) -
> Dict[str, Any]:
        """Simple retry."""
        import time
        time.sleep(2)  # Brief pause before retry
        return self._route_by_intent(classified)

    def _retry_with_credentials(self, classified: ClassifiedI
ntent) -> Dict[str, Any]:
        """Retry after user provided credentials."""
        if not self._pending or self._pending.get("stage") !=
"awaiting_credentials":
            return self._delegate_to_controller(classified.or
iginal_message)

        original_classified: ClassifiedIntent = self._pending
["original_classified"]

        # Merge credentials into entities
        entities = original_classified.entities.copy()
        entities["credentials"] = classified.entities

        updated = ClassifiedIntent(
            intent=original_classified.intent,
            confidence=original_classified.confidence,
            entities=entities,
            missing_required=[],
            clarify_question=None,
            original_message=original_classified.original_mes
sage,
        )
```

```
        self._pending = None
        return self._route_by_intent(updated)

    # ---------------------------------------------------------
-----------
    # Helpers
    # ---------------------------------------------------------
-----------

    def _delegate_to_controller(self, message: str) -> Dict[s
tr, Any]:
        """Delegate request to controller."""
        result = self.controller.handle_request(message)
        return self._wrap_result(result)

    def _wrap_result(
        self,
        result: Dict[str, Any],
        classified: ClassifiedIntent = None) -> Dict[str, An
y]:
        """Wrap controller result with standard fields."""
        return {
            "path": "FAST",
            "handled": result.get("handled", False),
            "response": result.get("response", ""),
            "tool_name": result.get("tool_name", ""),
            "tool_result": result.get("tool_result"),
            "steps": result.get("steps", []),
            "classified_intent": classified.intent if classif
ied else None,
            "entities": classified.entities if classified els
e None,
        }

    def _get_context(self) -> Dict[str, Any]:
```

```
        """Get current context for intent classification."""
        context = {}

        if self._pending:
            stage = self._pending.get("stage", "")
            if stage == "awaiting_clarification":
                context["awaiting"] = "clarification"
                context["pending_task"] = self._pending.get
("classified", {})
            elif stage == "awaiting_solution":
                context["awaiting"] = "solution_choice"
                context["last_error"] = str(self._pending.get
("interpretation", {}).get("original_error", ""))[:200]

        return context

    def _handle_pending(self, user_message: str) -> Dict[str,
Any]:
        """Handle message when there's pending state."""
        # Classify the new message
        context = self._get_context()
        classified = self.intent_classifier.classify(user_mes
sage, context)

        # Route appropriately
        return self._route_by_intent(classified)

    def _execute_pending_plan(self) -> Dict[str, Any]:
        """Execute a confirmed plan."""
        if not self._pending or self._pending.get("stage") !=
"awaiting_confirmation":
            return {
                "path": "FAST",
                "handled": True,
                "response": "Нечего подтверждать.",
                "tool_name": "no_pending",
```

```
            }

        plan = self._pending.get("plan", [])
        message = self._pending.get("original_message", "")
        self._pending = None

        # Execute plan through slow path
        return self._slow_path(message, plan)

    def _slow_path(self, message: str, plan: List = None) ->
Dict[str, Any]:
        """Execute multi-step plan."""
        if plan is None:
            plan = self.planner.plan(message)

        results = self.executor.execute_plan(plan)
        verification = self.verifier.verify(results)

        # Check for failures
        failed = [r for r in results if not r.success and not
r.skipped]
        if failed:
            return self._handle_step_failure(failed[0], messa
ge)

        return {
            "path": "SLOW",
            "handled": True,
            "response": verification.summary,
            "tool_name": "plan_executed",
            "tool_result": {
                "steps": len(results),
                "success": verification.ok,
            },
        }
```

```
    def _handle_step_failure(self, step: StepResult, original
_message: str) -> Dict[str, Any]:
        """Handle a failed step in plan execution."""
        context = {"task": original_message}
        interpretation = self.error_interpreter.interpret(ste
p.error or "", context)

        return {
            "path": "SLOW",
            "handled": True,
            "response": self.error_interpreter.format_for_use
r(interpretation),
            "tool_name": "step_failed",
            "tool_result": {"step": step.step_index, "error":
step.error},
            "awaiting_solution": interpretation.can_auto_fix,
        }

    # ---------------------------------------------------------
-----------
    # Learning
    # ---------------------------------------------------------
-----------

    def _load_learned_rules(self) -> None:
        """Load learned rules from file."""
        try:
            content = self._rules_file.read_text(encoding="ut
f-8")
            self._learned_rules = [
                line.strip() for line in content.splitlines()
                if line.strip().startswith("[")
            ]
        except FileNotFoundError:
            self._learned_rules = []
```

```
    def save_learned_rule(self, context: str, error: str, fi
x: str, rule: str) -> None:
        """Save a new learned rule."""
        entry = f"[{date.today()}] [{context}] → [{error}] →
[{fix}] → [{rule}]\n"
        with open(self._rules_file, "a", encoding="utf-8") as
f:
            f.write(entry)
        self._learned_rules.append(entry.strip())
Copy
```

---

3. Что изменить в `agent_v3.py`

Минимальные изменения — просто передать `memory_dir` при создании BrainLayer:

```
Copy# В функции main(), где создаётся BRAIN:

if CONTROLLER is not None:
    try:
        from brain.brain_layer import BrainLayer
        BRAIN = BrainLayer(
            CONTROLLER,
            require_confirmation=False,
            memory_dir=Path(MEMORY_DIR)  # <-- добавить
        )
        print("  BrainLayer: enabled (semantic classifier)")
    except Exception as e:
        BRAIN = None
        print(f"  BrainLayer: disabled ({e})")
```

Итого

Теперь вместо:

```
Copy# Старый подход (regex)
if "автоматиз" in msg or "workflow" in msg:
    intent = "CREATE_WORKFLOW"
```

Работает:

```
Copy# Новый подход (LLM)
classified = classifier.classify("хочу чтобы каждый день мне
приходили новости из телеги")
# -> intent: CREATE_WORKFLOW
# -> entities: {source: "telegram", schedule: "daily", destin
ation: "user"}
# -> missing: ["channels"]
# -> clarify_question: "Из каких телеграм-каналов собирать?"
```

Преимущества:

- Понимает любые формулировки
- Сам определяет чего не хватает и спрашивает
- Кэширует частые паттерны для скорости
- Fallback на эвристику если LLM недоступен

Перезапусти агент и попробуй разные формулировки — он должен понимать их все.